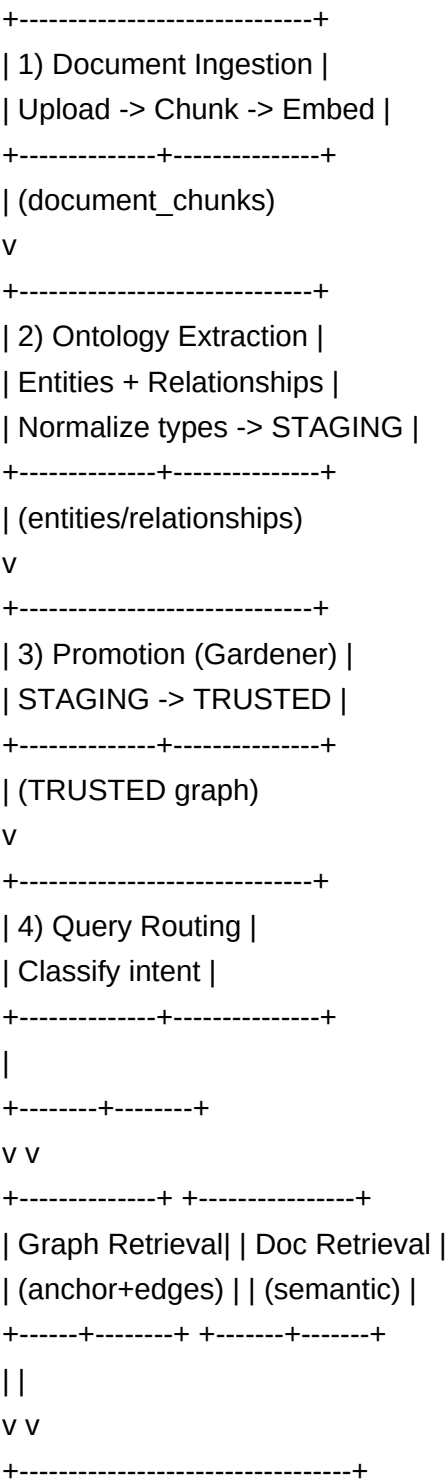


Context Foundry: Functional Architecture (Top-Down)

Purpose

Context Foundry answers enterprise questions by combining documents, a knowledge graph, and safety rules that prevent wrong answers. The system is designed to prefer "insufficient data" over confident errors and to generalize across corpora.

System Diagram



```

| 5) Conflict & Grounding |
| Entity grounding, conflict resolve|
+-----+-----+
|
v
+-----+
| 6) Metrics & Aggregation |
| Normalize metric/time -> compute |
+-----+
|
v
+-----+
| 7) Answer Synthesis |
| Cite evidence or "insufficient" |
+-----+

```

End-to-End Logic (Functional Flow)

1. Ingestion

- Documents are uploaded to a vault.
- Documents are chunked and embedded.
- `document\_chunks` is the authoritative input for extraction.

2. Extraction (Entities)

- LLM extracts entity candidates (name, type, properties).
- Types are normalized to ontology types.
- Guardrails correct obvious type errors.
- Entities are created with lifecycle `STAGING`.

3. Extraction (Relationships)

- LLM extracts relations between known entities.
- Relationship types are normalized to ontology types.
- Guardrails filter co-occurrence mistakes and type mismatches.
- Relationships are created with lifecycle `STAGING`.

4. Promotion (Gardener)

- Promotion thresholds move facts from `STAGING` to `TRUSTED`.
- If Gardener is not run, the graph remains untrusted and retrieval fails.

5. Routing & Retrieval

- Query is classified by intent.
- Retrieval chooses graph traversal, doc search, or both.
- Graph traversal uses the TRUSTED graph when available.

6. Conflict Resolution & Grounding

- Grounding checks the answer actually connects to the query entity.
- Conflicts are resolved by source trust, confidence, recency, and evidence.
- If contested, return NULL or fall back to documents.

7. Metric Normalization & Aggregation

- Normalize metric names and time periods.
- Classify facts as conflict vs complement.
- Compute aggregations programmatically (no LLM math).

8. Synthesis

- Answer with evidence and confidence.
- Prefer ?insufficient data? over low-confidence guesses.

Detailed Examples

Example 1: Role Query

- ? Question: Who is the CEO of Nexus Industries?
- ? Entity extraction finds PERSON + ORG, relation HOLDS_POSITION.
- ? Gardener promotes TRUSTED facts.
- ? Graph retrieval finds CEO candidates.
- ? Conflict resolution picks highest-confidence trusted candidate.
- ? Answer is returned with provenance.

Example 2: Supplier Query

- ? Question: Who supplies solar panels for Desert Sun?
- ? Extraction creates SUPPLIES edge between supplier and project.
- ? Graph retrieval uses project anchor ? SUPPLIES edges.
- ? Grounding ensures supplier is linked to Desert Sun.
- ? Answer: First Solar, with evidence.

Example 3: Aggregation

- ? Question: Total value of top 3 customer relationships.
- ? Extraction yields per-customer contract values.
- ? Canonicalize metric names and periods.
- ? Resolve duplicates per customer.
- ? Rank by value, sum top 3 programmatically.
- ? Answer with coverage and sources.

Failure Modes (System-Level)

- ? Missing chunks ? extraction sees ?no content.?
- ? Unmapped relationship types ? candidate store only.
- ? No promotion ? graph remains 100% STAGING.
- ? Weak relationship coverage ? low recall.
- ? Tree ON unintentionally ? noisy/slow retrieval.
- ? Supply-chain edges missing ? supplier queries fail.

Diagnostic Principles

- ? Compare old vs new vault structure: chunks, entities, relationships.
- ? Check lifecycle distribution: TRUSTED vs STAGING.
- ? Confirm tree flag state at runtime.
- ? Validate vault ID in result files after each test.

One-Page Architecture Brief

Context Foundry converts documents into a trusted knowledge graph and answers questions with retrieval + conflict-aware logic. The system flow is:

Upload ? Chunk ? Extract Entities/Relationships ? Normalize ? Stage ? Promote ? Retrieve ? Resolve ? Aggregate ? Answer

Reliability depends on three upstream guarantees:

- ? Chunks exist before extraction.
- ? Relationships normalize to ontology types.
- ? STAGING facts are promoted to TRUSTED.

If those conditions hold, retrieval and conflict resolution can operate correctly. If they fail, downstream logic collapses regardless of prompt quality or retrieval algorithms.

Files

- ? This document: `docs/architecture\_summary.md`